

06 — Vector Embeddings & the Embedding Provider Pattern

What is an embedding?

Think of an embedding as a **fingerprint for meaning**. When you read the sentence "industrial machinery and steel structures", your brain understands the *meaning* — it's about manufacturing, metal, heavy industry. An embedding captures that same understanding as a list of numbers.

```
"industrial machinery" → [0.013, 0.020, 0.005, -0.059, ... ] (768 numbers)
"ipari gépgyártás"      → [0.012, 0.019, 0.006, -0.058, ... ] (768 numbers – very similar!)
"chocolate cake recipe" → [0.891, -0.342, 0.117, 0.445, ... ] (768 numbers – very different)
```

The key insight: **texts with similar meaning get similar numbers**. The Hungarian translation of "industrial machinery" produces nearly identical numbers to the English version, because they mean the same thing. "Chocolate cake recipe" produces completely different numbers, because it means something different.

Why 768 numbers?

Each number represents one "dimension" of meaning. You can think of it like coordinates:

- 2D coordinates (x, y) can place you on a flat map
- 3D coordinates (x, y, z) can place you in a room
- 768D coordinates can place text in a "meaning space" with 768 axes

More dimensions = more nuance. With 768 dimensions, the model can capture subtle differences between "steel manufacturing" and "steel trading" — they're close in meaning-space but not identical.

Different models produce different numbers of dimensions:

Provider	Model	Dimensions
Gemini	gemini-embedding-001	768 (configurable via MRL)
Jina AI	jina-embeddings-v3	1024
Local BGE	BAAI/bge-m3	1024

How similarity search works

Once text is converted to number-lists, finding "similar" content becomes pure math:

```
Query: "acélszerkezet gyártó" (steel structure manufacturer)
↓ embed
Query vector: [0.013, 0.020, ...]
↓ compare against all stored vectors using cosine similarity
Result: closest vectors = most relevant records
```

Cosine similarity measures the angle between two vectors. If they point in the same direction (angle $\approx 0^\circ$), the texts mean similar things. If they point in perpendicular directions (angle $\approx 90^\circ$), they're unrelated.

This is why we installed **pgvector** — it adds vector math to PostgreSQL. Our HNSW index (from learning doc 02) makes this comparison fast even with thousands of records.

The Embedding Provider Pattern

Just like we built a Strategy Pattern for LLM providers (doc 05), we built the same pattern for embeddings:

```
EmbeddingProvider (ABC)
■■■ GeminiEmbedding    – free, 768 dims, Google API
■■■ JinaEmbedding      – 10M free tokens, 1024 dims
■■■ LocalBGEEmbedding – runs on your machine, no API needed, 1024 dims
```

The interface is simple:

```
class EmbeddingProvider(ABC):
    name: str = "base"
    dimensions: int = 768

    @abstractmethod
    def embed(self, texts: list[str]) -> list[list[float]]:
        ...

    def embed_single(self, text: str) -> list[float]:
        return self.embed([text])[0]
```

Every provider takes text in, numbers out. The caller never knows which API is doing the work.

Why three providers?

Provider	Pros	Cons	When to use
Gemini	Free, good quality, configurable dimensions	Requires internet, rate limits	Default — primary choice
Jina	10M free tokens, excellent multilingual support	Tokens deplete over time	Fallback if Gemini is down
Local BGE	Zero cost, works offline, no rate limits	Slow, downloads 2GB model, needs GPU for speed	Offline development, or when APIs are unavailable

Batching: one text at a time vs. all at once

An important implementation detail — different APIs handle batching differently:

Gemini — one embedding per API call:

```
def embed(self, texts):
    results = []
    for text in texts:
        # loop: one call per text
        resp = requests.post(url, json={
            "content": {"parts": [{"text": text}]},
```

```

        "outputDimensionality": 768
    })
    results.append(resp.json()["embedding"]["values"])
    return results

```

Jina — all texts in one API call:

```

def embed(self, texts):
    resp = requests.post(url, json={
        "input": texts,                # batch: all texts at once
        "dimensions": 1024
    })
    return [item["embedding"] for item in resp.json()["data"]]

```

Batching is faster (one network round-trip instead of many), but Gemini's embedding API only accepts one text per request. This is hidden behind the interface — callers just pass a list and get a list back.

MRL: Matryoshka Representation Learning

Gemini's embedding model supports **MRL** — you can request fewer dimensions than the model's maximum. Think of it like a Russian nesting doll: the first 256 dimensions already capture the most important meaning, and each additional dimension adds finer detail.

```

payload = {
    "outputDimensionality": 768    # could be 256, 512, 768, etc.
}

```

We use 768 because:

1. Our `category_vectors` and `company_vectors` tables have `VECTOR(768)` columns
2. 768 dims gives good quality for bilingual content without using too much storage
3. It matches the lower dimension count (Jina/BGE use 1024, but we'd need to truncate or pad)

The Prompt Store: dynamic system prompts

One more piece of Phase C — the **PromptStore**. Instead of hardcoding the system prompts for translation and enrichment, we store them in the database:

```

class PromptStore:
    def __init__(self):
        self._cache = {}
        self._load_from_db()    # reads from app_config table

    def get_translation_prompt(self):
        return self._cache.get("prompt_translation") or DEFAULT_TRANSLATION_PROMPT

```

Why this matters: The Admin UI will have a text area where you can edit system prompts. When you change the prompt and re-run the ETL, the new prompt takes effect immediately — no code deployment needed. This satisfies requirement R24.

The fallback to `DEFAULT_TRANSLATION_PROMPT` ensures the system works even if the database is empty or unreachable.

How it all connects

```
Admin UI dropdown: "Gemini" ■■→ app_config DB table
                        ↓
ETL starts ■■→ get_embedding_provider() reads DB ■■→ creates GeminiEmbedding
                        ↓
                    "ipari gépgyártás" ■■→ [0.013, 0.020, ...] (768 floats)
                        ↓
                Stored in company_vectors.embedding column
                        ↓
                Search API queries pgvector for similarity
```

Key vocabulary

Term	Meaning
Embedding	A list of numbers (vector) that represents the meaning of text — similar meanings produce similar numbers
Dimensions	How many numbers are in the embedding — more dimensions capture more nuance
Cosine similarity	A math formula that measures how similar two vectors are by comparing the angle between them
HNSW	An index that makes vector similarity search fast — trades a tiny bit of accuracy for huge speed gains
MRL	Matryoshka Representation Learning — ability to use fewer dimensions while keeping the most important meaning
Batch embedding	Sending multiple texts in one API call for efficiency
Prompt Store	A pattern where system prompts are stored in the database and loaded at runtime, making them editable without code changes

Test yourself

1. Why do similar-meaning texts in different languages produce similar embedding vectors?
2. If we switched from Gemini (768 dims) to Jina (1024 dims), what else would we need to change?
3. Why does the PromptStore cache prompts in memory instead of reading the database on every call?
4. What happens if the Gemini API is down and we haven't set up Jina or local BGE? How would you handle this?
5. Why is the local BGE provider useful even though it's slower than the API-based ones?